Dominic Sitto CMPSC 448 Midterm Project

Problem Definition & Dataset:

We are tasked with finding if you can use the output of multiple LLMs to correctly identify which one wrote which, as well as seeing how the user's input can help us make the prediction, possibly without looking at the output at all.

Gathered dataset from the Large Model Systems Organization's Chatbot Arena on Kaggle: https://www.kaggle.com/competitions/lmsys-chatbot-arena/

This dataset shows the result from their own arena they ran, in which they give a single prompt to two LLMs and have a human judge which of the LLMs did a better job. While this does not completely align with our problem, this is a somewhat recent (2 years) dataset that is very large (around 57000 rows). To make this better match our goal, we split every row into two, and removed the information relating to the judges, this way we just have the model used, the prompt given, and the response returned. This doubled the amount of data we have to around 110000 responses, then we decided to only look at 3 specific models, it would be unnecessarily complex to classify the response into 64 different models, we took 3 of the most common models: gpt-4-0613, claude-2.1 & llama-2-70b-chat with 6165, 5583 & 3428 responses respectively. We split this data into 20% testing and 80% training, later splitting the training into a further 80% training, 20% validation, resulting in: 64% training, 16% validation and 20% test

Lastly we also tokenized the inputs using keras library's TextVectorization with a whitespace split and standardization via lower_and_strip_punctuation. We set Max tokens to 20000 and sequence_length to 1000. When using both LLM_input and LLM_output as inputs, we simply attached them together into one string. Additionally, when training the models on both LLM_input and LLM_output we tested setting sequence_length to 5000 in case the double string was too large, however there was no substantial impact on the model's results.

```
def createTokenizer(trainingSet):
    vectorizer = tf.keras.layers.TextVectorization(
        max_tokens=MAX_TOKENS,
        standardize="lower_and_strip_punctuation",
        split="whitespace",
        output_mode="int",
        output_sequence_length=SEQUENCE_LENGTH,
    )

    vectorizer.adapt(
        tf.data.Dataset.from_tensor_slices(trainingSet).batch(64)
    )
    return vectorizer
```

This provides us with the processed data we need to run the models.

CNN Implementation:

The CNN models include:

- Input layer taking a single string of input for each item
- Text vectorization layer tokenizing the inputs of the strings
- Embedding layer of 128 output dimensions
- One dimensional convolutional layer with filters of either 64, 128 or 256 with kernel size of 3, 5 or 7 using ReLU activation without any added padding
- Global max-pooling layer
- Dropout layer of 50%
- Hidden dense layer of 64 neurons with ReLU activation
- Output dense layer classifying into one of 3 possible LLMs using softmax

Lastly we use an Adam optimizer with learning rates: 3e-4, 1e-3 or 3e-3 and with loss function sparse_categorical_crossentropy. The model has a keras monitor on val_loss with patience 3 while restoring best weights

For each type of input: LLM_Output, LLM_Input and LLM_Output + LLM_Input we tested each combination of filters, kernel sizing and learning rates.

```
def trainModelCNN(vectorizer, kernelSize, lr, filters):
    inputs = tf.keras.Input(shape=(1,), dtype=tf.string, name="text")

    x = vectorizer(inputs)

    x = tf.keras.layers.Embedding(
        input_dim=len(vectorizer.get_vocabulary()),
        output_dim=128,
    )(x)

    x = tf.keras.layers.Conv1D(
        filters=filters,
        kernel_size=kernelSize,
        activation="relu",
        padding="valid",
    )(x)

    x = tf.keras.layers.GlobalMaxPooling1D()(x)
    x = tf.keras.layers.Dropout(0.5)(x)
    x = tf.keras.layers.Dense(64, activation="relu")(x)

    outputs = tf.keras.layers.Dense(
        3,
        activation="softmax",
    )(x)

    model = tf.keras.Model(inputs, outputs)

    model.compile(
```

```
        optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )

    # model.summary()
    return(model)
```

RNN Implementation

The RNN models include:

- Input layer with a single string for each item
- Text vectorization layer
- Embedding layer of output dimensions of either 64 or 128 with padding on 0s
- LSTM layer with 64 units
- Dropout layer of 50%
- Hidden dense layer (not always implemented) with 64 values and ReLU activation
- Output dense layer of 3 possible LLM results using softmax activation

Lastly we use an Adam optimizer with learning rates: 3e-4 or 3e-3 and with loss function sparse_categorical_crossentropy. The model has a keras monitor on val_loss with patience 3 while restoring best weights

For each type of input: LLM_Output, LLM_Input and LLM_Output + LLM_Input we tested each combination of enabling/disabling the dense layer, output dimensions and learning rates

```
def trainModelRNN(vectorizer, enabledDense, lr, output_dim):
    inputs = tf.keras.Input(shape=(1,), dtype=tf.string, name="text")

    x = vectorizer(inputs)

    x = tf.keras.layers.Embedding(
        input_dim=len(vectorizer.get_vocabulary()),
        output_dim=output_dim,
        mask_zero=True,
    )(x)

    x = tf.keras.layers.LSTM(
        units=64,
        return_sequences=False
    )(x)

    x = tf.keras.layers.Dropout(0.5)(x)
    if (enabledDense):
        x = tf.keras.layers.Dense(64, activation="relu")(x)

    outputs = tf.keras.layers.Dense(
        3,
```

```
        activation="softmax",
    )(x)

    model = tf.keras.Model(inputs, outputs)

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )

    # model.summary()
    return(model)
```

Results:

Note: we are going to assume 40.623% accuracy as the null hypothesis as if you always guess gpt-4-0613 as the model (the most common LLM in this dataset) you will have 40.623% accuracy.

We present the highest accuracy result of each set of tested hyperparameters for each type of neural network.

Input condition	CNN test accuracy	CNN macro F1	RNN test accuracy	RNN macro F1
Output only	84.35%	0.83	76.90%	0.75
Input only	41.02%	0.27	40.63%	0.19
Input + output	82.57%	0.81	75.91%	0.74

Training on just LLM_Output data

We found that while both the CNN and RNN performed substantially better than the null hypothesis, the CNN had greater success.

Specifically with the CNN, there was not a substantial variation in results from modifying the hyperparameters to say definitively if changing them had a significant effect on the model, however 5, 256 and 0.0003 were the highest performing ones. While there was starting to be overfitting across the training set, the validation caught it and the validation set had a very similar accuracy to the test set. Possibly with a larger data set we could reach a validation and test accuracy closer to the training set. When looking at the confusion matrix we can also see that it is much more common to have false positives predicting ChatGPT over other LLMs, this is interesting because while ChatGPT is the most common LLM in the data set it is only 10% more common than Claude. This is possible evidence that while Claude and Llama can have substantially different outputs, ChatGPT hits more of a middle ground between the two.

Using the RNN model there was also not a major difference in enabling or disabling the hidden dense layer or changing output dimensions, however the smaller learning rate was substantially more accurate as well as even training slightly faster. The overall spread of predictions was very similar to that of the CNN, showing that the model is skewed or biased towards ChatGPT. It is also important to note that training the RNN takes substantially longer than the CNNs, this stays consistent throughout the rest of the tests.

Overall we can predict the LLM model that generated the response just based on its output with an accuracy substantially higher than just always guessing the most common model, even without seeing the actual input the user wrote. This also shows that some aspect of ChatGPT makes the model falsely predict ChatGPT disproportionately more than other LLMs.

CNN with just output data:

Kernel size: 5 Filters: 256 Learning rate: 0.0003

```
304/304 ━━━━━━━━━━━━━━━━━━━━ 38s 121ms/step - accuracy: 0.5202 - loss: 0.9789 -
val_accuracy: 0.6628 - val_loss: 0.7809
Epoch 2/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 36s 119ms/step - accuracy: 0.7143 - loss: 0.6465 -
val_accuracy: 0.7872 - val_loss: 0.5253
Epoch 3/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 36s 120ms/step - accuracy: 0.8113 - loss: 0.4532 -
val_accuracy: 0.8234 - val_loss: 0.4542
Epoch 4/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 37s 122ms/step - accuracy: 0.8825 - loss: 0.3115 -
val_accuracy: 0.8345 - val_loss: 0.4059
Epoch 5/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 37s 121ms/step - accuracy: 0.9357 - loss: 0.1933 -
val_accuracy: 0.8530 - val_loss: 0.3931
Epoch 6/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 36s 117ms/step - accuracy: 0.9625 - loss: 0.1187 -
val_accuracy: 0.8489 - val_loss: 0.4048
Epoch 7/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 36s 118ms/step - accuracy: 0.9768 - loss: 0.0778 -
val_accuracy: 0.8547 - val_loss: 0.4209
Epoch 8/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 36s 117ms/step - accuracy: 0.9867 - loss: 0.0480 -
val_accuracy: 0.8481 - val_loss: 0.4471

Test accuracy: 0.8434925675392151
                  precision    recall  f1-score   support

      claude-2.1       0.92      0.88      0.90      1116
      gpt-4-0613       0.79      0.88      0.84      1233
llama-2-70b-chat       0.82      0.71      0.76       686

        accuracy                           0.84      3035
       macro avg       0.85      0.83      0.83      3035
    weighted avg       0.85      0.84      0.84      3035

[[ 980  117   19]
 [  54 1091   88]
 [  27  170  489]]
```

RNN with just output data

Hidden dense layer: True Output dimensions: 128 Learning rate: 0.0003

```
304/304 ━━━━━━━━━━━━━━━━━━━━ 86s 275ms/step - accuracy: 0.5036 - loss: 0.9906 -
val_accuracy: 0.6501 - val_loss: 0.7960
Epoch 2/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 83s 274ms/step - accuracy: 0.7512 - loss: 0.5808 -
val_accuracy: 0.7822 - val_loss: 0.5592
Epoch 3/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 83s 274ms/step - accuracy: 0.8298 - loss: 0.4713 -
val_accuracy: 0.7089 - val_loss: 0.6749
Epoch 4/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 83s 273ms/step - accuracy: 0.9040 - loss: 0.2733 -
val_accuracy: 0.7711 - val_loss: 0.6394
Epoch 5/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 83s 272ms/step - accuracy: 0.9500 - loss: 0.1491 -
val_accuracy: 0.7797 - val_loss: 0.7150

Test accuracy: 0.7690280079841614
                  precision    recall  f1-score   support

      claude-2.1       0.87      0.82      0.85      1116
      gpt-4-0613       0.70      0.83      0.76      1233
llama-2-70b-chat       0.75      0.57      0.65       686

        accuracy                           0.77      3035
       macro avg       0.77      0.74      0.75      3035
    weighted avg       0.77      0.77      0.77      3035

[[ 920  178   18]
 [  95 1025  113]
 [  42  255  389]]
```

Training on just LLM_Input

Unlike using just the LLM_Output data we could not substantially surpass the null hypothesis with either the CNN or the RNN.

The CNN did get very slightly over the null hypothesis but many other attempts were below, this could be attributed to random luck. The CNNs presented very little change over the different hyper parameters. The CNN completely gave up on trying to predict Llama and instead just focused on Claude and ChatGPT but was only very slightly better at predicting Claude than if it just stuck to ChatGPT. It is also interesting to note that despite completely dropping a column, it took 15 seconds longer each epoch to train than training on LLM_Output.

The RNNs had the best success when they just reached the Null Hypothesis, a few different sets of other hyperparameters looked closer to the CNN with a few guesses towards Claude however, none were better than just only predicting ChatGPT. This also took longer to train than the LLM_Output set. It is interesting to note that as further training of the model immediately lead to overfitting, being a very clear indicator that the dataset is not fit for just using LLM_Input as our input.

Overall with this data set we cannot predict which model an input was targeted towards. This is possibly due to the manner of the dataset. This dataset is based on a test in which multiple LLMs were compared to each

other, it is very common for the same or similar input to be shared by multiple different LLMs. In the way we initially processed the dataset we split each individual input into two items for each LLM used. Possibly if data was gathered simply on just organic users of the LLMs we could have a better idea of how to predict LLM based on just the input.

CNN with just input data

Kernel size: 7 Filters: 256 Learning rate: 0.003

```
304/304 ━━━━━━━━━━━━━━━━━━━━ 48s 155ms/step - accuracy: 0.3927 - loss: 1.0747 -
val_accuracy: 0.4014 - val_loss: 1.0721
Epoch 2/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 47s 155ms/step - accuracy: 0.4177 - loss: 1.0495 -
val_accuracy: 0.3882 - val_loss: 1.0805
Epoch 3/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 47s 154ms/step - accuracy: 0.5149 - loss: 0.9420 -
val_accuracy: 0.3660 - val_loss: 1.1410
Epoch 4/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 47s 155ms/step - accuracy: 0.6421 - loss: 0.7687 -
val_accuracy: 0.3594 - val_loss: 1.2827

Test accuracy: 0.4102141559123993
                  precision    recall  f1-score   support

      claude-2.1       0.42      0.18      0.25      1116
      gpt-4-0613       0.41      0.85      0.55      1233
llama-2-70b-chat       0.00      0.00      0.00       686

        accuracy                           0.41      3035
       macro avg       0.28      0.34      0.27      3035
    weighted avg       0.32      0.41      0.32      3035

[[ 203  913    0]
 [ 191 1042    0]
 [  89  597    0]]
```

RNN with just input data

Hidden dense layer: True Output dimensions: 128 Learning rate: 0.0003

```
304/304 ━━━━━━━━━━━━━━━━━━━━ 139s 453ms/step - accuracy: 0.4054 - loss: 1.0748 -
val_accuracy: 0.4063 - val_loss: 1.0699
Epoch 2/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 133s 439ms/step - accuracy: 0.4084 - loss: 1.0625 -
val_accuracy: 0.4051 - val_loss: 1.0723
Epoch 3/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 136s 448ms/step - accuracy: 0.4711 - loss: 1.0059 -
val_accuracy: 0.3549 - val_loss: 1.1186
Epoch 4/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 139s 456ms/step - accuracy: 0.5912 - loss: 0.8542 -
```

```
val_accuracy: 0.3503 - val_loss: 1.2375

Test accuracy: 0.40626028180122375
                  precision    recall  f1-score   support

      claude-2.1       0.00      0.00      0.00      1116
      gpt-4-0613       0.41      1.00      0.58      1233
llama-2-70b-chat       0.00      0.00      0.00       686

        accuracy                           0.41      3035
       macro avg       0.14      0.33      0.19      3035
    weighted avg       0.17      0.41      0.23      3035

[[   0 1116    0]
 [   0 1233    0]
 [   0  686    0]]
```

Training on both LLM_Input and LLM_Output

We found a slight decrease in accuracy when combining LLM_Input with LLM_Output as opposed to just training from the LLM_Output.

The CNN matched the same pattern we found before in just LLM_Output being biased towards the ChatGPT LLM. We also found the time to train was about as long as it was with just the LLM_Input.

The RNN similarly matched its previous results using just the LLM_Output. However it took substantially less time to run each Epoch than the LLM_Input models, however it had substantially more epochs.

Overall adding the LLM_Input to the LLM_Output slightly reduces accuracy as well as both increasing our required data as well as the resources required to train the models.

There was also a worry that because both strings are added to each other, we may surpass the limit set in our vectorization or tokenization, we increased the sequence_length from 1000 to 5000. This had no impact on results at all.

CNN with both input and output data

Kernel size: 7 Filters: 256 Learning rate: 0.001

```
304/304 ━━━━━━━━━━━━━━━━━━━━ 49s 160ms/step - accuracy: 0.6416 - loss: 0.7633 -
val_accuracy: 0.7867 - val_loss: 0.5271
Epoch 2/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 49s 161ms/step - accuracy: 0.8374 - loss: 0.4024 -
val_accuracy: 0.8283 - val_loss: 0.4307
Epoch 3/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 49s 161ms/step - accuracy: 0.9422 - loss: 0.1601 -
val_accuracy: 0.8172 - val_loss: 0.5433
Epoch 4/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 49s 160ms/step - accuracy: 0.9790 - loss: 0.0691 -
val_accuracy: 0.8254 - val_loss: 0.5970
Epoch 5/20
```

```
304/304 ━━━━━━━━━━━━━━━━━━━━ 53s 174ms/step - accuracy: 0.9895 - loss: 0.0368 -
val_accuracy: 0.8213 - val_loss: 0.6819

Test accuracy: 0.8257001638412476
                  precision    recall  f1-score   support

      claude-2.1       0.94      0.84      0.89      1116
      gpt-4-0613       0.76      0.91      0.82      1233
llama-2-70b-chat       0.80      0.66      0.72       686

        accuracy                           0.83      3035
       macro avg       0.83      0.80      0.81      3035
    weighted avg       0.84      0.83      0.83      3035

[[ 935  152   29]
 [  31 1117   85]
 [  24  208  454]]
```

RNN with both input and output data

Hidden dense layer: True Output dimensions: 64 Learning rate: 0.0003

```
304/304 ━━━━━━━━━━━━━━━━━━━━ 105s 341ms/step - accuracy: 0.4943 - loss: 1.0189 -
val_accuracy: 0.6357 - val_loss: 0.8210
Epoch 2/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 105s 347ms/step - accuracy: 0.7071 - loss: 0.6608 -
val_accuracy: 0.7081 - val_loss: 0.6454
Epoch 3/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 105s 347ms/step - accuracy: 0.8540 - loss: 0.3813 -
val_accuracy: 0.7678 - val_loss: 0.5881
Epoch 4/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 105s 346ms/step - accuracy: 0.9265 - loss: 0.2188 -
val_accuracy: 0.7736 - val_loss: 0.6817
Epoch 5/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 105s 344ms/step - accuracy: 0.9563 - loss: 0.1374 -
val_accuracy: 0.7645 - val_loss: 0.7660
Epoch 6/20
304/304 ━━━━━━━━━━━━━━━━━━━━ 103s 339ms/step - accuracy: 0.9725 - loss: 0.0884 -
val_accuracy: 0.7559 - val_loss: 0.8548

Test accuracy: 0.7591433525085449
                  precision    recall  f1-score   support

      claude-2.1       0.85      0.83      0.84      1116
      gpt-4-0613       0.70      0.82      0.76      1233
llama-2-70b-chat       0.73      0.53      0.62       686

        accuracy                           0.76      3035
       macro avg       0.76      0.73      0.74      3035
    weighted avg       0.76      0.76      0.76      3035
```

```
[[ 921  166   29]
 [ 110 1017  106]
 [  57  263  366]]
```

Lessons & Experience

Difference in LLMs

After this project, we have learned that there are significant differences in how different LLMs might respond textually to different or similar inputs. It would be interesting to do further research and see exactly which phrases are more likely to come from specific LLMs or even to take it further and see that certain LLMs are more likely to delusionally support the user or more likely to pass on advertisements to users. While we could not come to a conclusion on whether the input a user provides can have correlation on which LLM they are using, I could still see that being a possibility. Perhaps Claude might be better at assisting in programming whereas ChatGPT is more desirable in writing emails.

Using CNNs and RNNs

We have found that RNNs are extremely resource heavy when training and especially when testing different combinations of hyperparams over many models, it can be extremely time consuming to train RNNs. CNNs however are substantially faster and in this case more successful. While most of the hyperparams had very minimal effect on the different models, the RNNs are very vulnerable to change in learning rate and picking the correct learning rate can both substantially improve accuracy but also speed up runtime. This was interesting to us because theoretically increasing learning rate sounds like it would speed up training time at the cost of accuracy, but with too large a learning rate, a model wouldn't be able to converge on its best set of weights. Additionally, using 256 filters in the convolutional layer consistently produced the highest accuracy across all three CNN input conditions. The RNNs also had the most success by including a hidden dense layer.